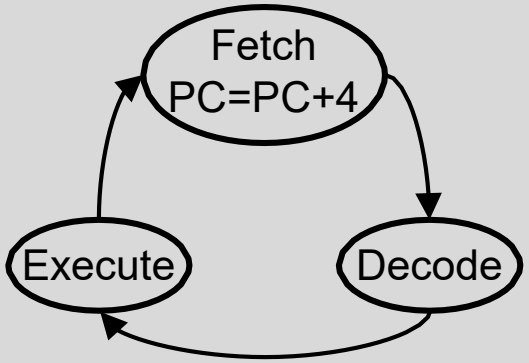
Computer System Architecture

Lecture 5 - Datapath

October 2026

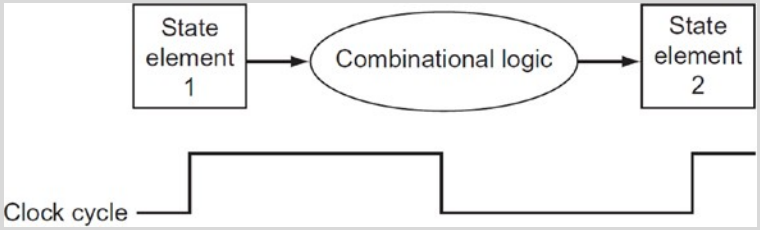
The Processor: Datapath and Control

- Our implementation of the MIPS is simplified
  - Memory-reference instructions: `lw`, `sw`
  - Arithmetic-logical instructions: `add`, `sub`, `and`, `or`, `slt`
  - Control flow instructions: `beq`, `j`
- Generic implementation
  - use the program counter (PC) to supply the instruction address and **fetch** the instruction from memory (and update the PC)
  - **decode** the instruction (and read registers)
  - **execute** the instruction
- All instructions (except `j`) use the ALU after reading the registers



Clocking Methodology

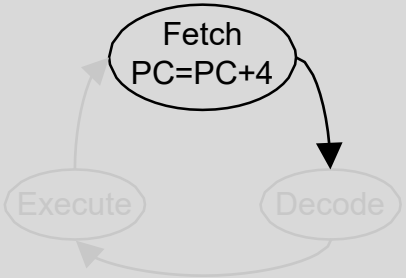
- The clocking methodology defines when data in a state element is valid and stable relative to the clock.
  - State elements: a memory element such as a register
  - Edge-triggered: all state changes occur on a clock edge
- Typical execution
  - read contents of state elements ➔ send values through combinational logic ➔ write results to one or more state elements



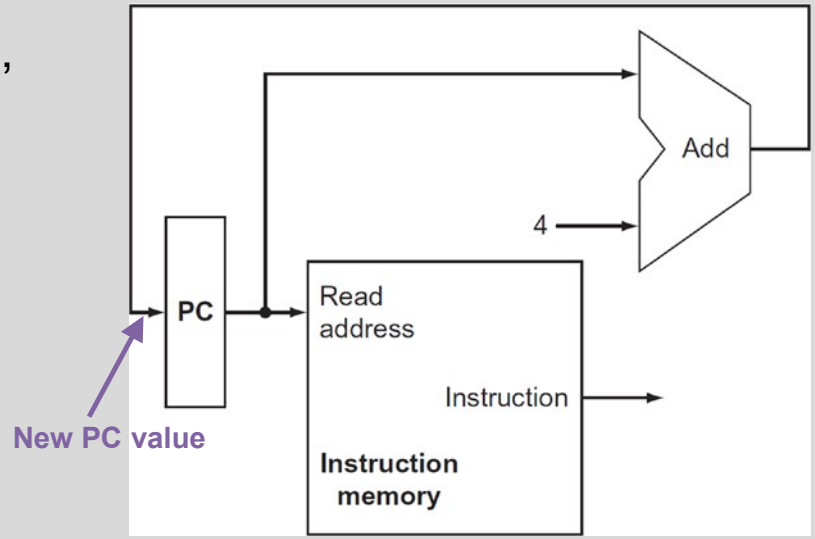
- Assumes state elements are written on every clock cycle; if not, need explicit write control signal
  - write occurs only when both the write control is asserted and the clock edge occurs

Fetching Instructions

- Fetching instructions involves
  - reading the instruction from the Instruction Memory
  - updating the PC value to be the address of the next (sequential) instruction

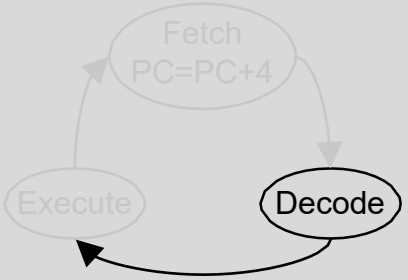


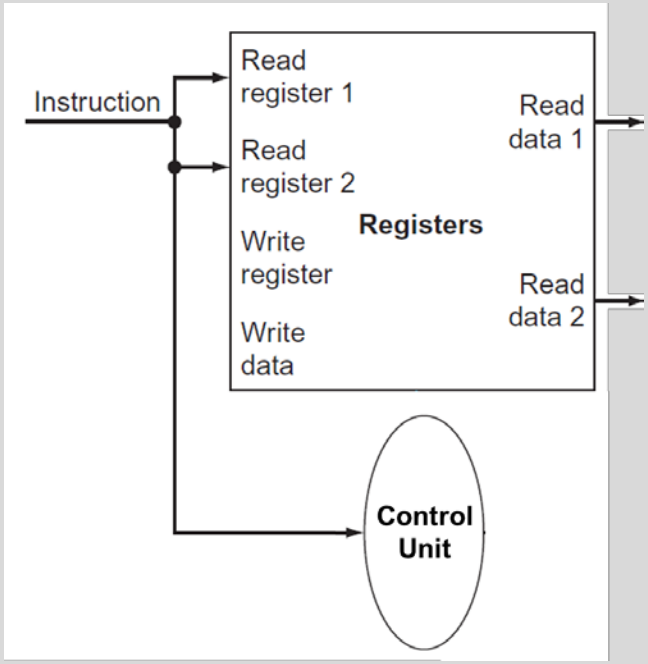
  - PC is updated every clock cycle, so it does not need an explicit write control signal just a clock signal
  - Reading from the Instruction Memory is a combinational activity, so it doesn't need an explicit read control signal



Decoding Instructions

- Decoding instructions involves
  - sending the fetched instruction's opcode and function field bits to the control unit
  - reading two values from the Registers
    - register addresses are contained in the instruction



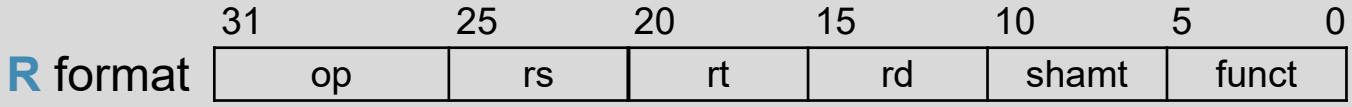


Reading Registers “Just in Case”

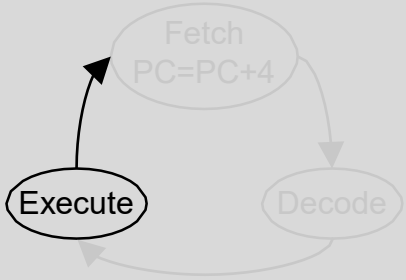
- Note that both read ports of the Registers are active for all instructions during the decode cycle, using the rs and rt instruction field addresses
  - Since haven’t decoded the instruction yet, don’t know what the instruction is.
  - Just in case the instruction uses values from the Registers do “work ahead” by reading the two source operands

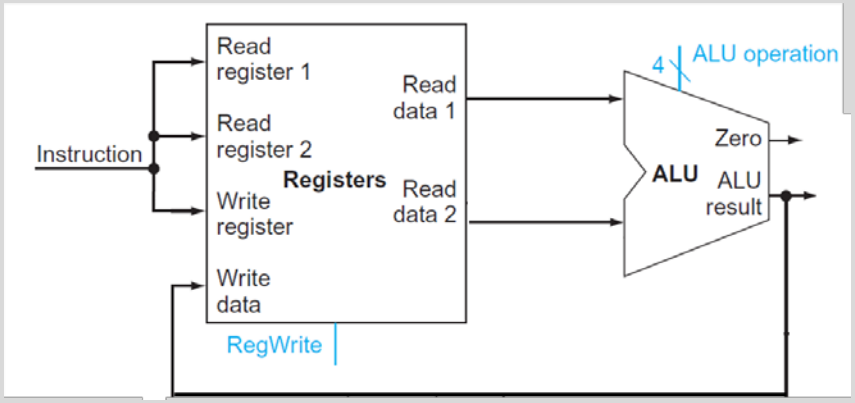
Executing R Format Operations

- R format operations (add, sub, slt, and, or)



  - perform operation (**op** and **funct**) on values in **rs** and **rt**
  - store the result back into the Registers (into location **rd**)

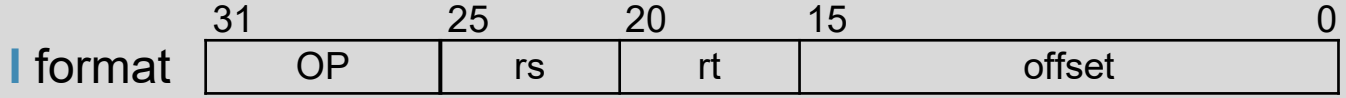




  - Note that register is not written every cycle (e.g. `SW`), so we need an explicit write control signal for the Registers

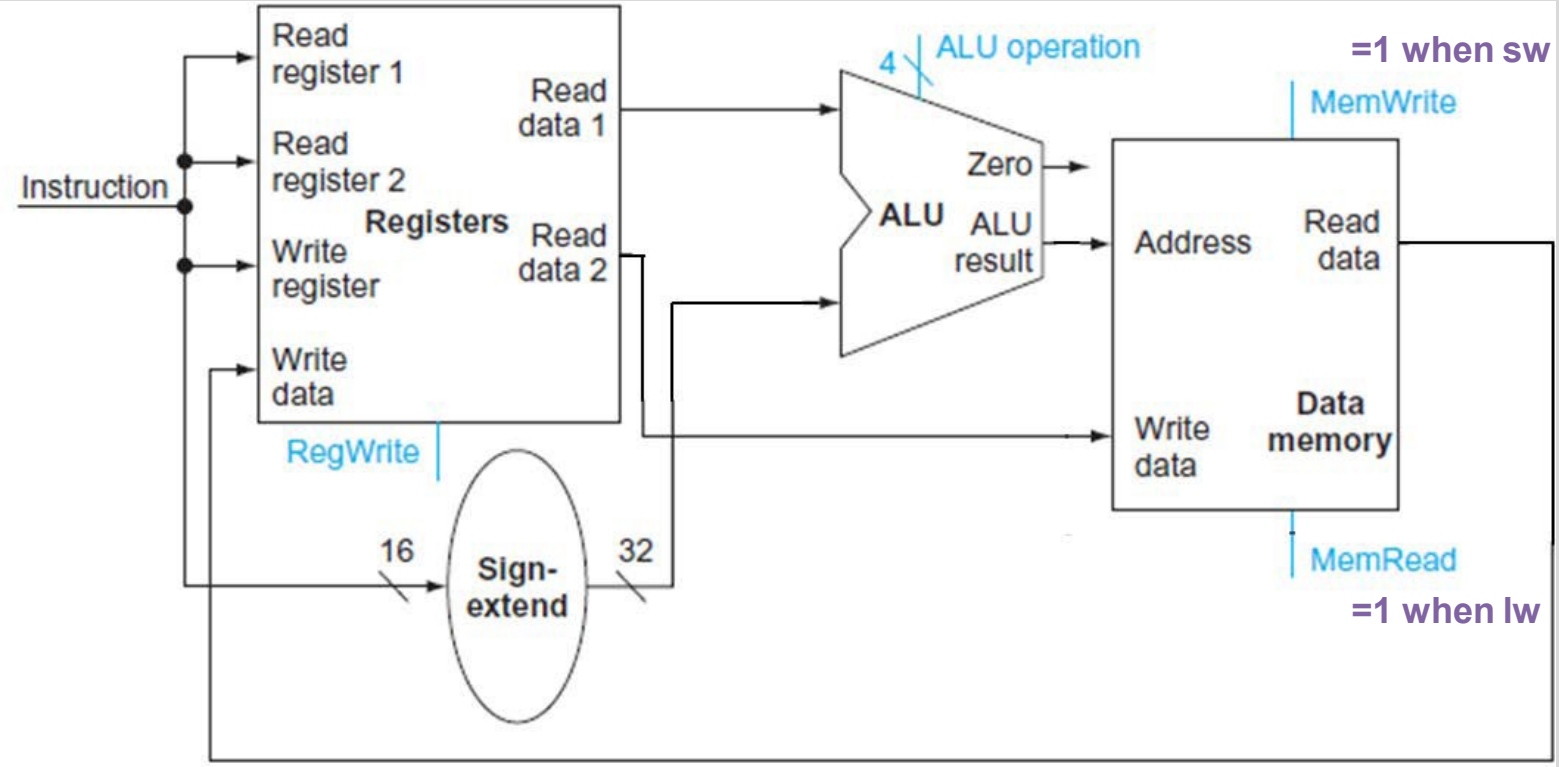
Executing Load and Store Operations

- Load and store operations have to



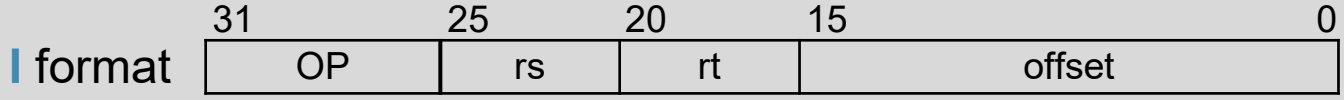
  - compute a memory address by adding the base register (in **rs**) to the 16-bit signed offset field in the instruction
    - base register was read from the Registers during decode
    - offset value in the low order 16 bits of the instruction, must be **sign-extended** to create a 32-bit signed value
  - **store** value, read from the Registers during decode, must be written to the Data Memory
  - **load** value, read from the Data Memory, must be stored in the Registers

Executing Load and Store Operations



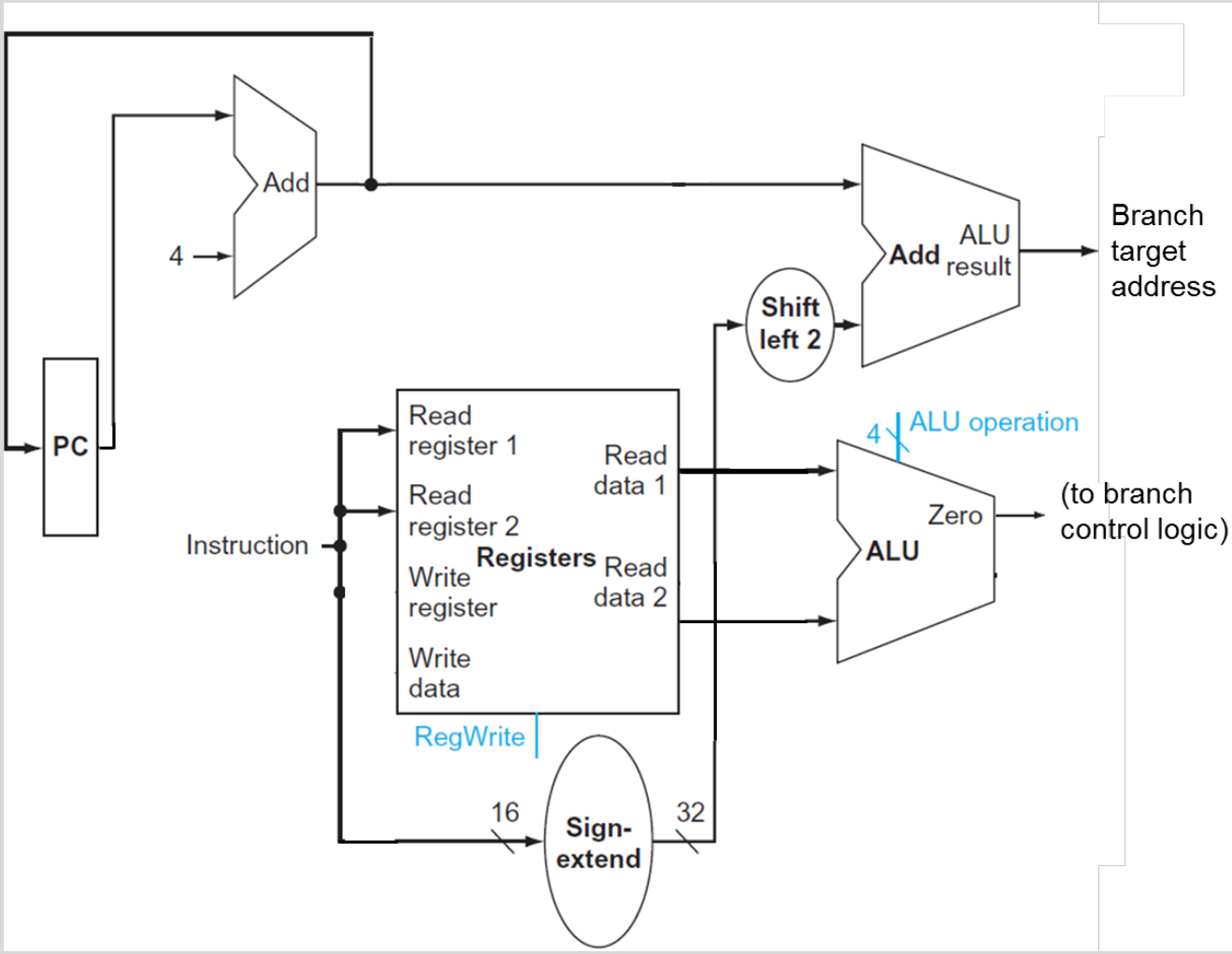
Executing Branch Operations

- Branch operations have to



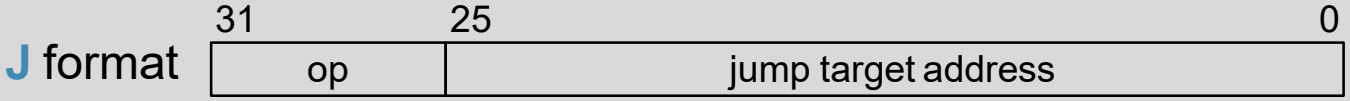
  - compare the operands read from the Registers during decode (**rs** and **rt** values) for equality (zero ALU output)
  - compute the branch target address by adding the updated PC to the sign-extended 16-bit offset field in the instruction
    - "base register" is the updated PC
    - offset value in the low order 16 bits of the instruction must be sign extended to create a 32-bit signed value and then shifted left 2 bits

Executing Branch Operations

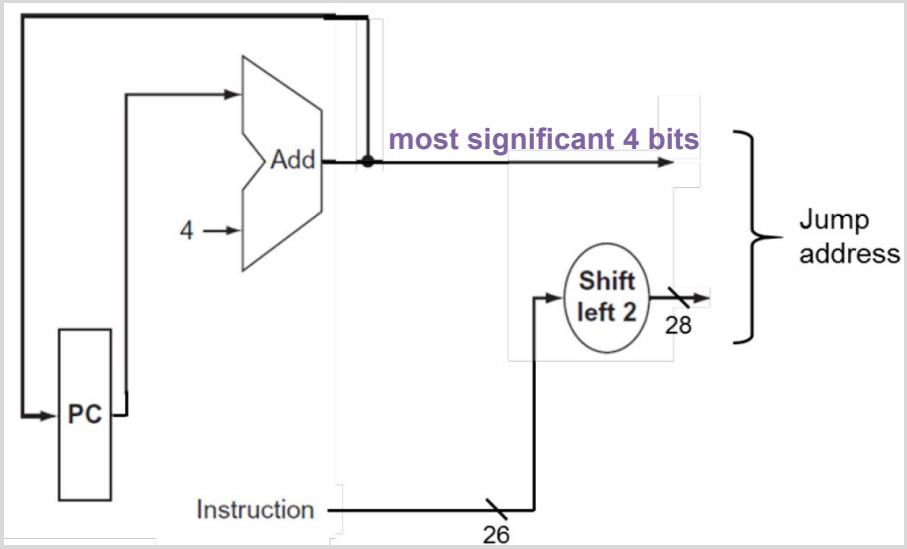


Executing Jump Operations

- Jump operations have to



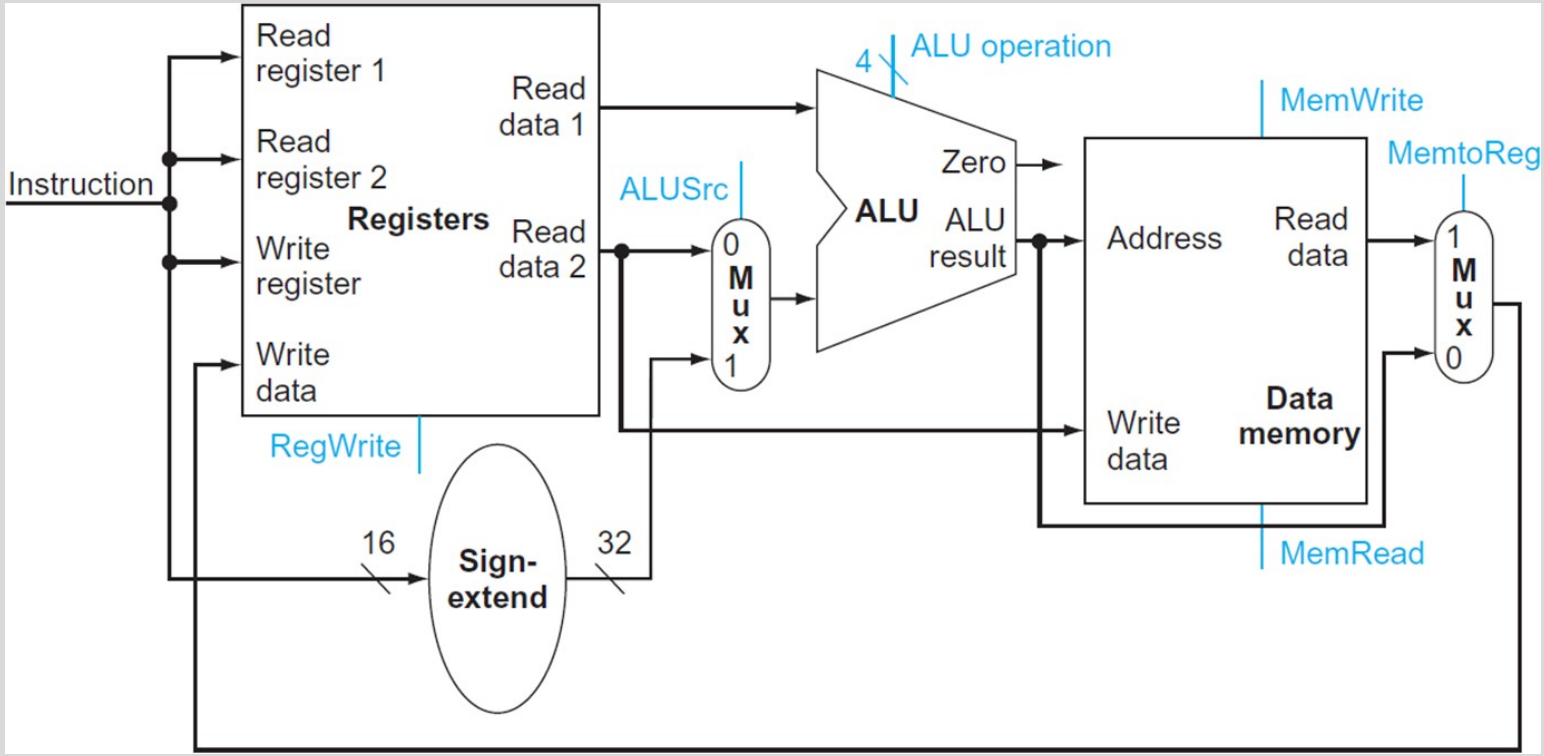
  - replace the lower 28 bits of the PC with the lower 26 bits of the fetched instruction shifted left by 2 bits



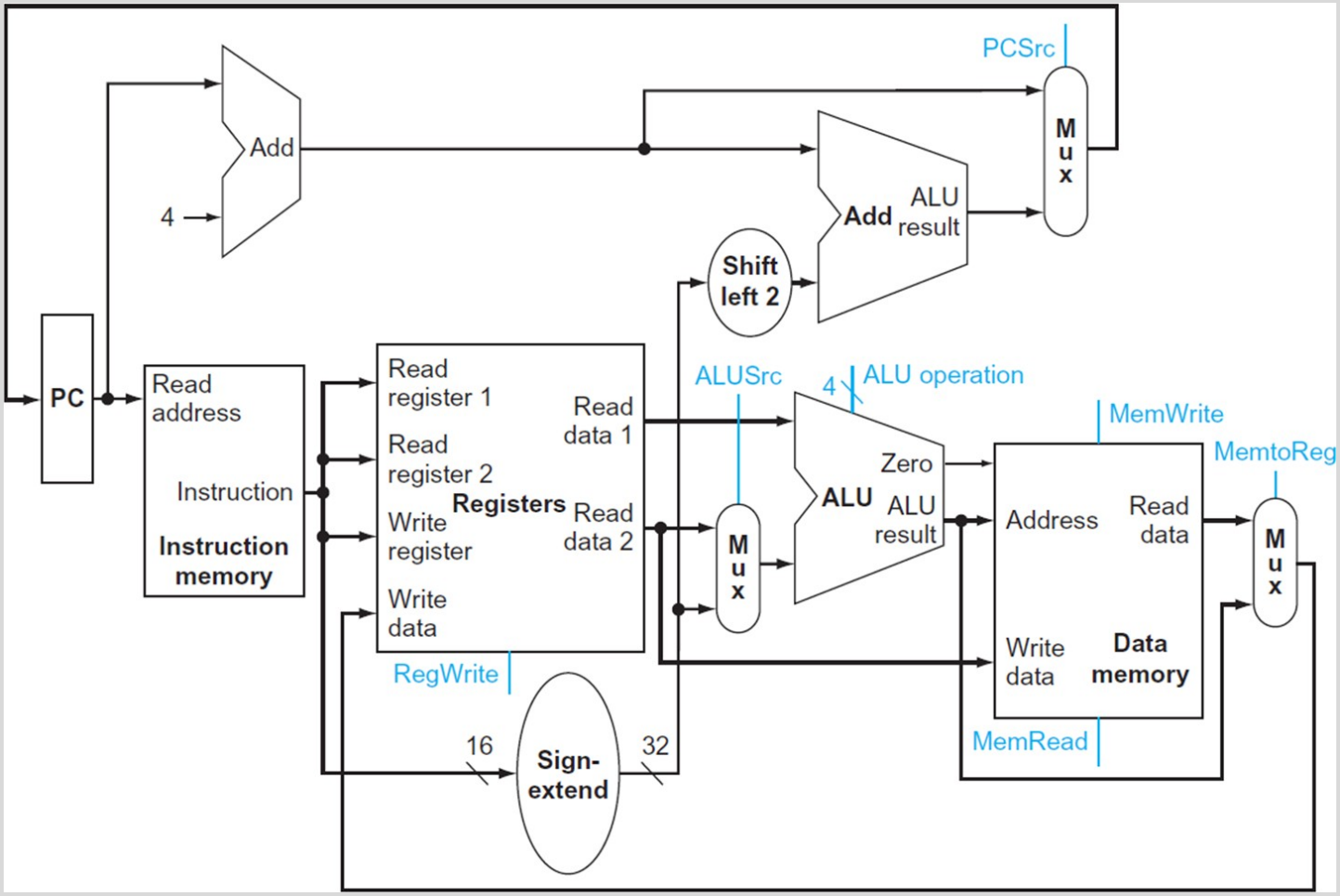
Creating a Single Datapath from the Parts

- Assemble the datapath segments and add control lines and multiplexors as needed
- **Single-cycle** design – fetch, decode and execute each instructions in one clock cycle
  - no datapath resource can be used more than once per instruction, so some must be **duplicated**
  - **Multiplexors** needed at the input of shared elements with control lines to do the selection
  - write signals to control writing to the Register File and Data Memory
- **Cycle time** is determined by length of the longest path

Datapath for R-type and Memory Instructions

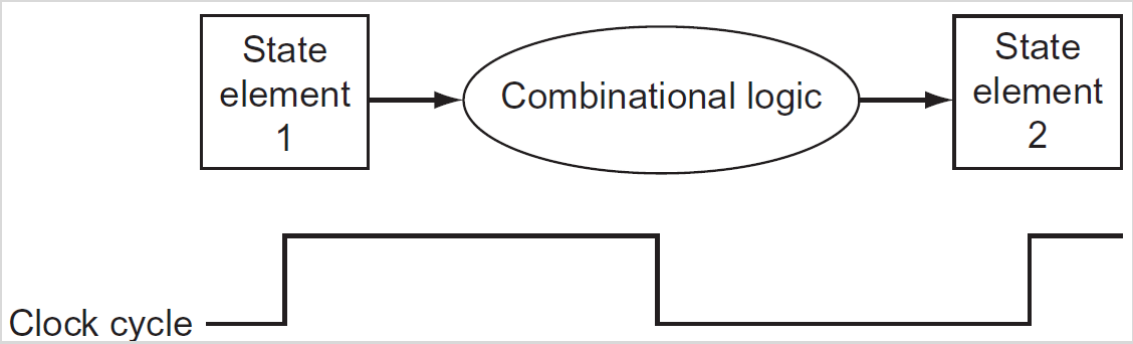


Adding Branch Instructions



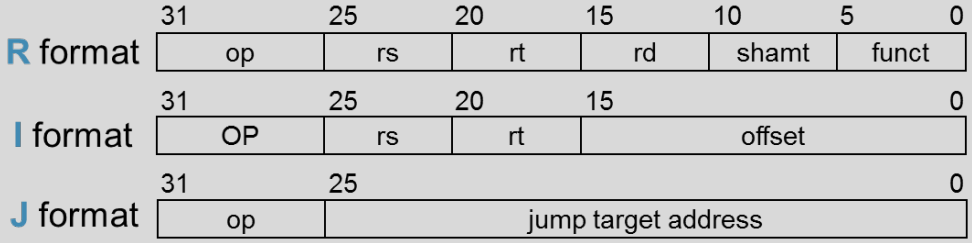
Our Simple Control Structure

- We wait for everything to settle down
  - ALU might not produce “right answer” right away
  - Combinational logic: ALU, adders, muxes, shifter, sign-extender
  - Use write signals along with the clock edge to determine when to write to sequential elements (PC, Registers, and Data Memory)
- The clock cycle time is determined by the logic delay through the longest path

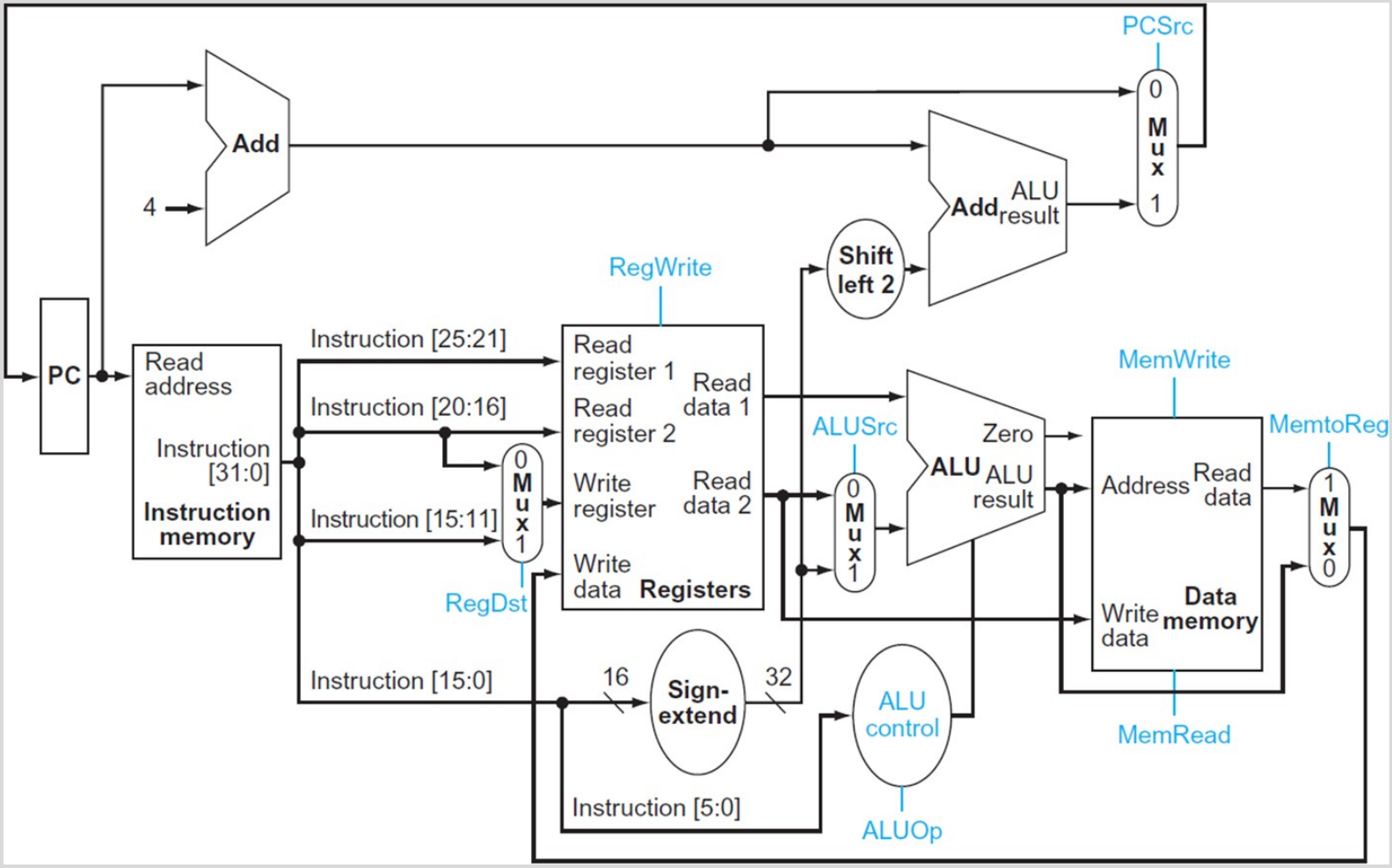


Adding the Control

- Selecting the operations to perform (ALU, Register and Memory read/write)
- Controlling the flow of data (multiplexor inputs)
- Information comes from the 32 bits of the instruction
- Observations
  - op field always in bits 31–26
  - addresses of two registers to be read are always specified by the rs and rt fields (bits 25–21 and 20–16)
  - base register for `lw` and `sw` always in rs (bits 25–21)
  - address of register to be written is in one of **two** places: in rt (bits 20–16) for `lw`; in rd (bits 15–11) for R-type instructions
  - offset for `beq`, `lw`, and `sw` always in bits 15–0



(Almost) Complete Single-Cycle Datapath



ALU Control

- ALU used for
  - Load/Store: add
  - Branch: subtract (compare in `beq`)
  - R-type: depends on funct field

ALU Control	Function
0000	AND
0001	OR
0010	add
0011	subtract
0111	set-on-less-than
1100	NOR

ALU Control

- Assume 2-bit **ALUOp** derived from opcode
  - Combinational logic derives ALU control

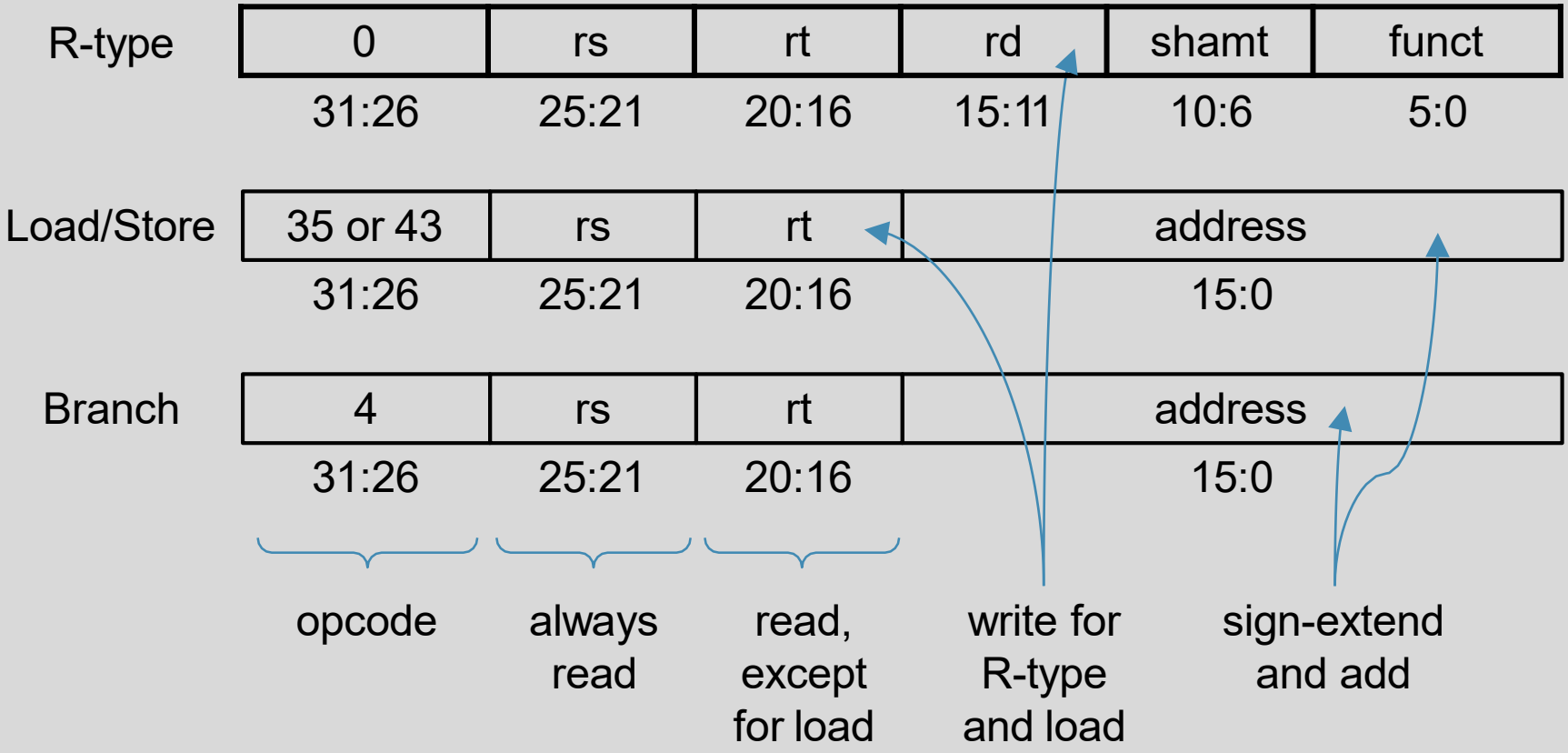
opcode	ALUOp	Opeation	funct	ALU function	ALU control
lw	00	load word	XXXXXX	add	0010
sw	00	store word	XXXXXX	add	0010
beq	01	branch equal	XXXXXX	subtract	0110
R-type	10	add	100000	add	0010
		subtract	100010	subtract	0110
		ADD	100100	AND	0000
		OR	100101	OR	0001
		set-on-less-than	101010	set-on-less-than	0111

Designing the Main Control Unit

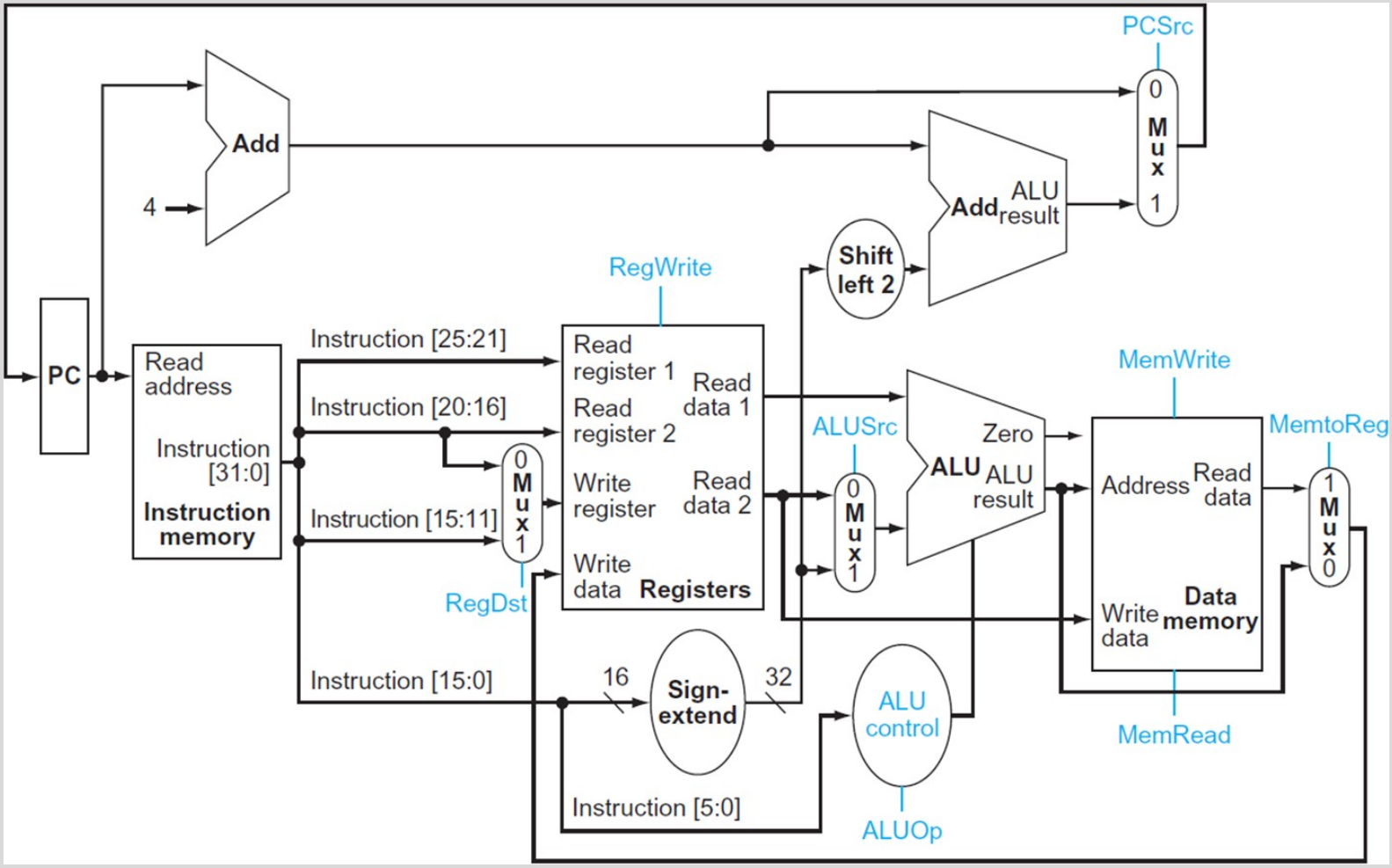
ALUOp		Funct field						Operation
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	0110
1	X	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	X	X	X	0	1	0	0	0000
1	X	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

The Main Control Unit

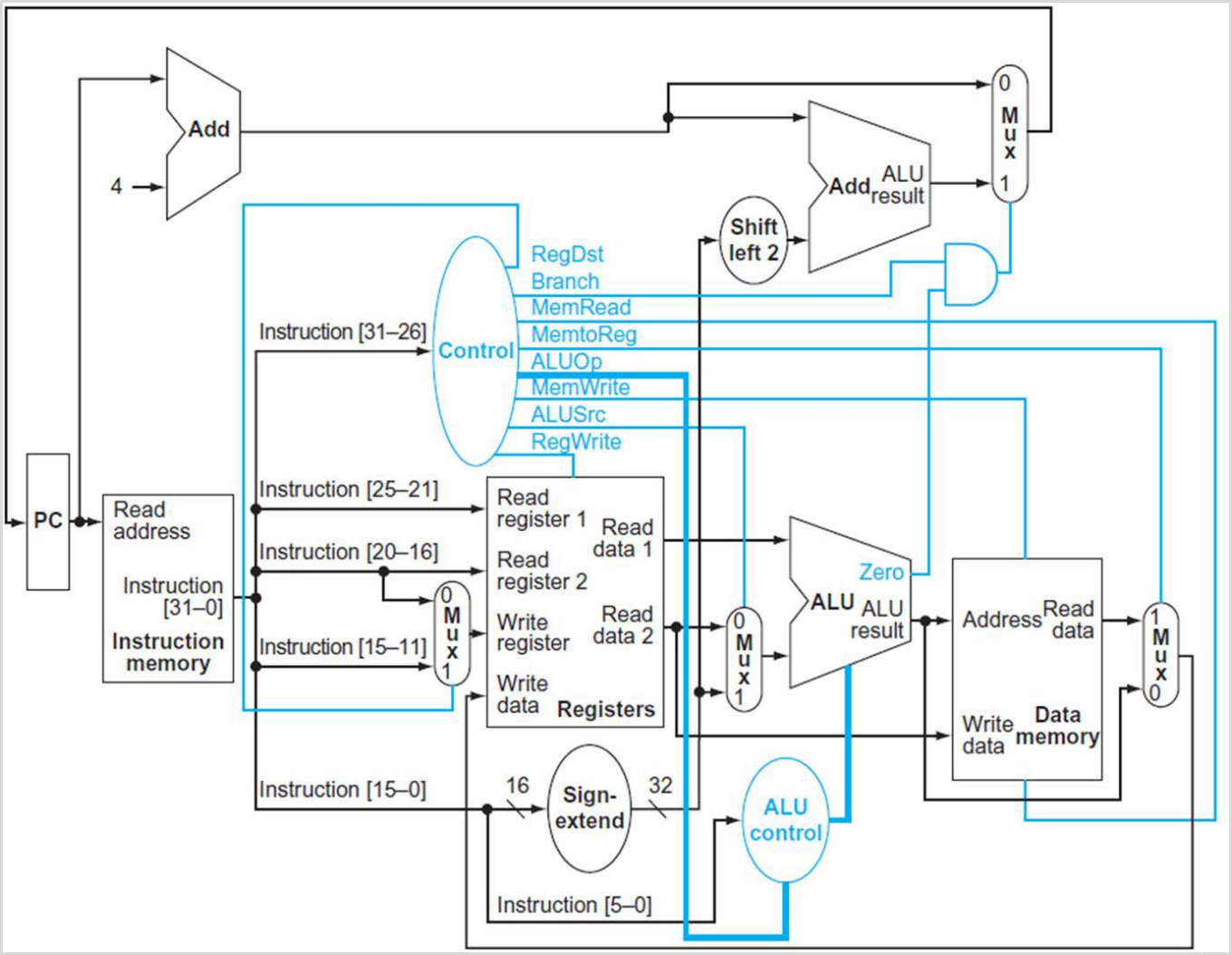
- Control signals derived from instruction



(Almost) Complete Single-Cycle Datapath

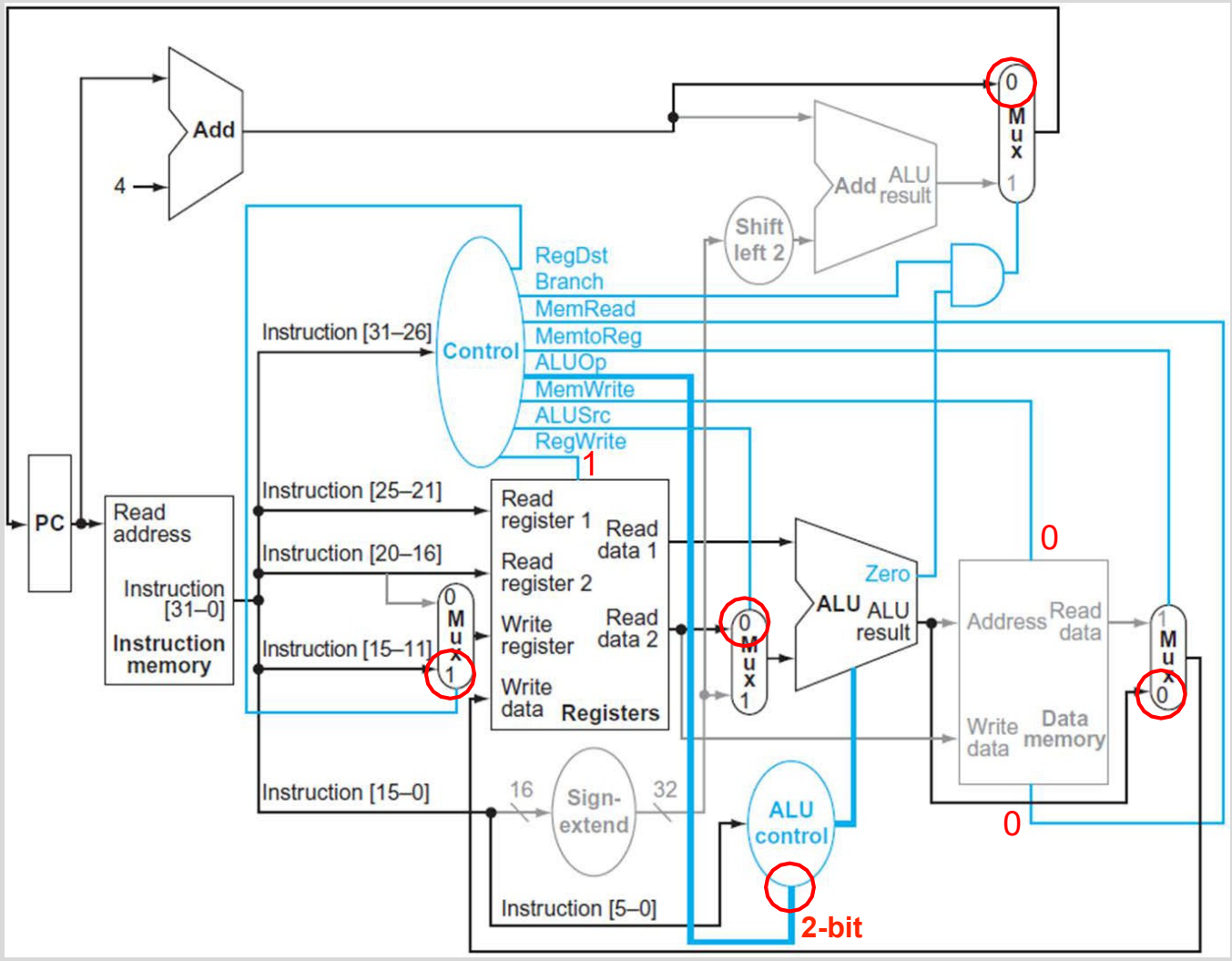


Datapath with Control

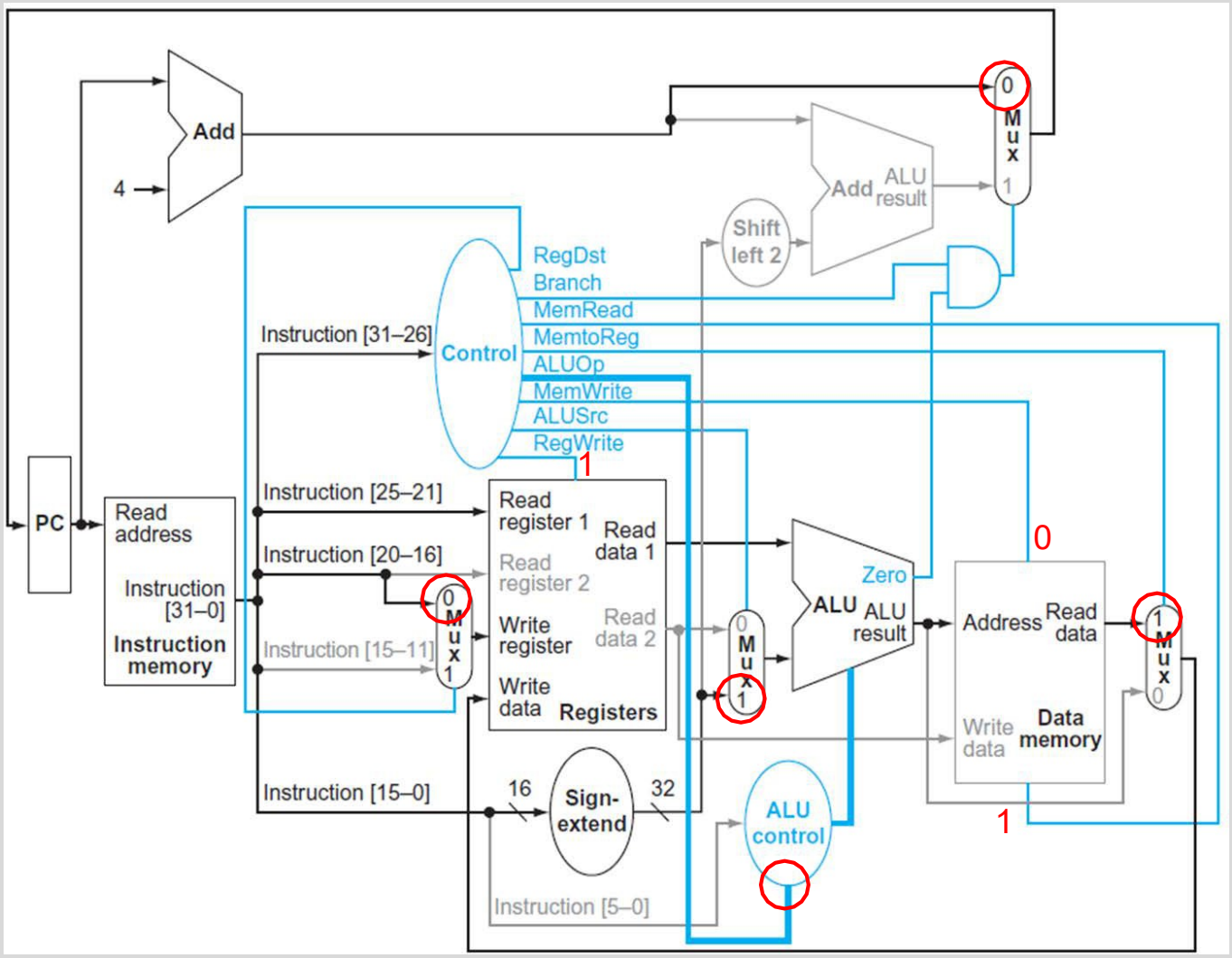


How to set control bits for R-type / load / beq instructions?

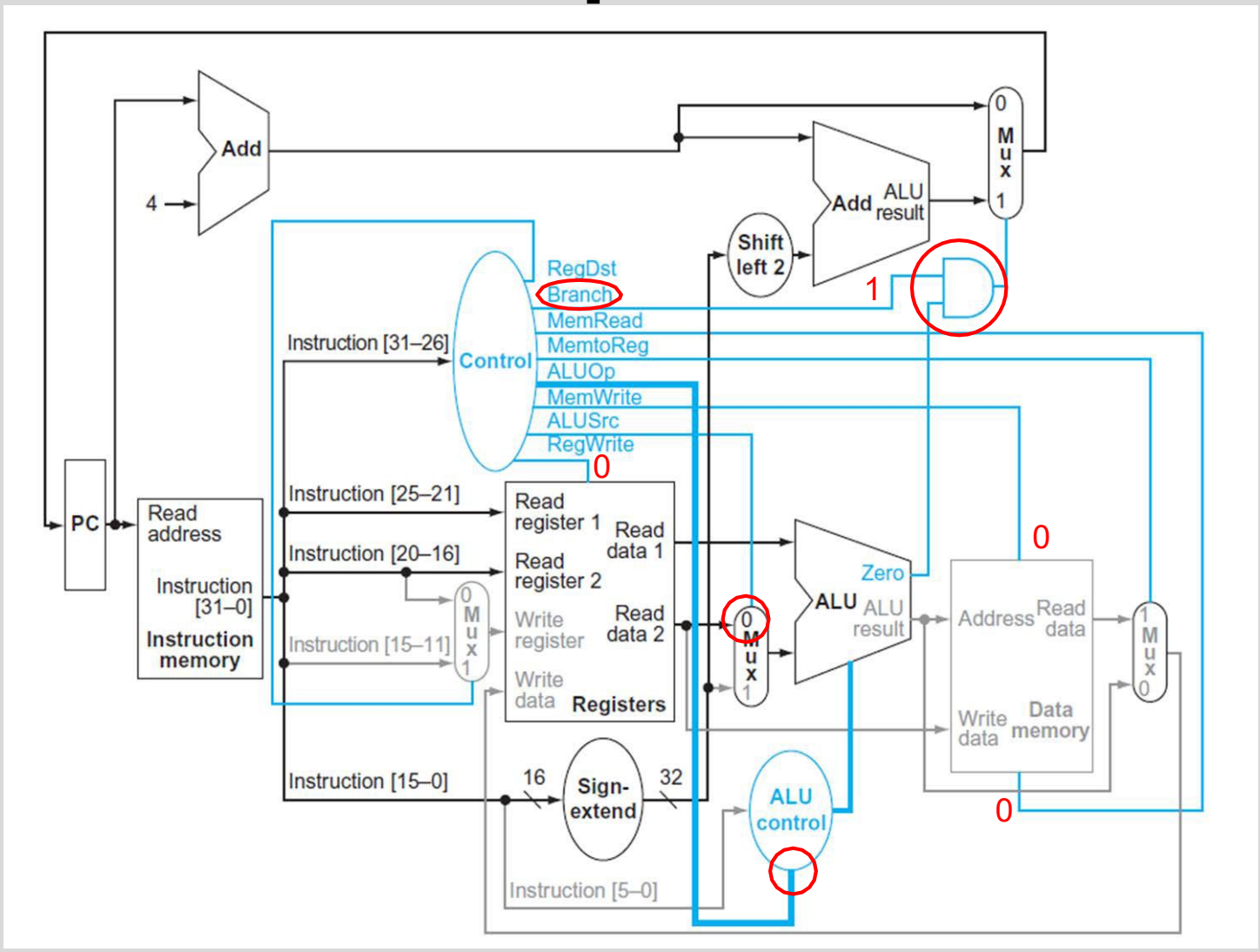
R-type Instruction



Load Instruction



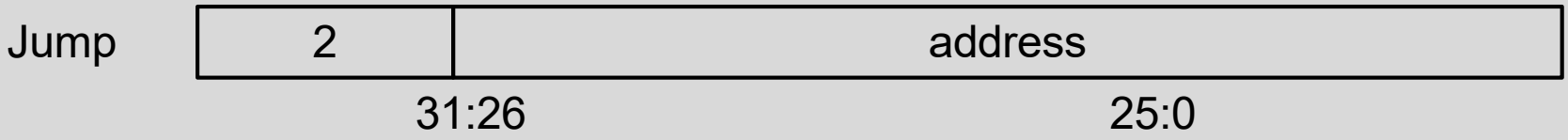
Branch-on-Equal Instruction



Control Lines

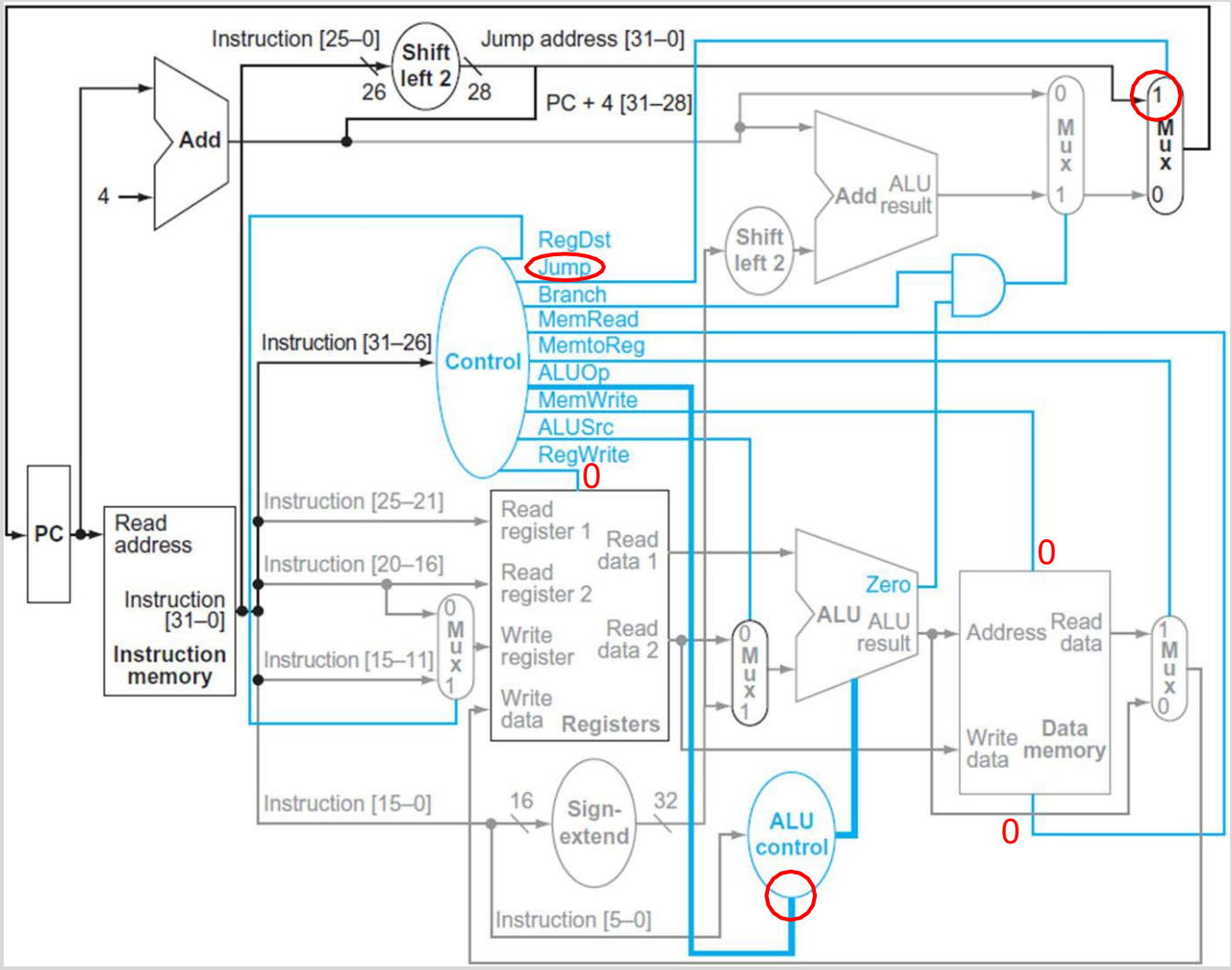
Instruction	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	MemWrite	Branch	ALUOp
R-type 000000	1	0	0	1	0	0	0	10
lw 100011	0	1	1	1	1	0	0	00
sw 101011	X	1	X	0	0	1	0	00
beq 000100	X	0	X	0	0	0	1	01

Implementing Jumps



- Jump uses word address
- Update PC with concatenation of
  - Top 4 bits of old PC
  - 26-bit jump address
  - 00 (shifting)
- Need an extra control signal decoded from opcode

Datapath With Jumps Added



Instruction Critical Paths

- Calculate cycle time assuming negligible delays (for muxes, control unit, sign extend, PC access, shift left 2, wires) except:
  - Instruction and Data Memory (4 ns)
  - ALU and adders (2 ns)
  - Register File access (reads or writes) (1 ns)

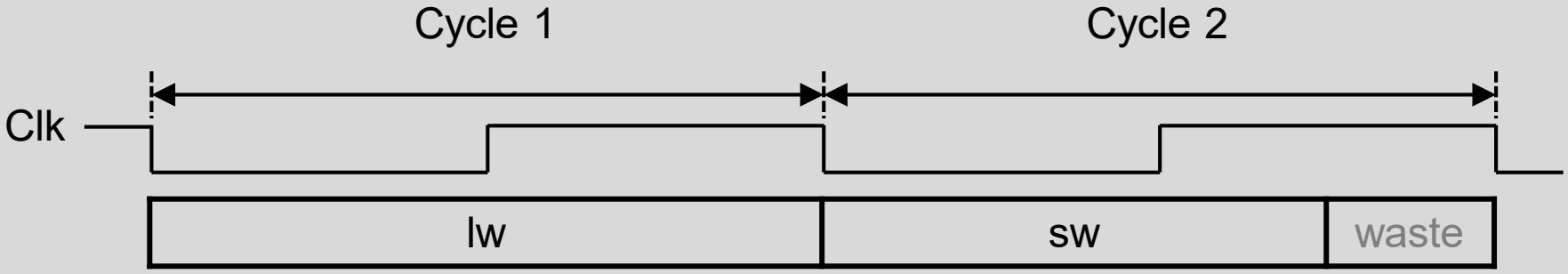
Instruction	I Mem	Reg Rd	ALU Op	D Mem	Reg Wr	Total
R-type	4	1	2		1	8
load	4	1	2	4	1	12
store	4	1	2	4		11
beq	4	1	2			7
jump	4					4

Performance Issues

- Longest delay determines clock period
  - Critical path: load instruction
  - Instruction Memory → Registers → ALU → Data Memory → Registers
- Not feasible to vary period for different instructions
- Violates design principle
  - Making the common case fast

Single Cycle Disadvantages & Advantages

- Uses the clock cycle inefficiently – the clock cycle must be timed to accommodate the slowest instruction



- Wasteful of area – some functional units (e.g., adders) must be duplicated
- Simple and easy to understand